

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

KHOA ĐIỆN - ĐIỆN TỬ



HỌC KỲ 2 NH 2025-2026

MÃ HỌC PHẦN: MPCA333364

LỚP HỌC PHẦN: 252MPCA333364_01

MÔN HỌC: KIẾN TRÚC MÁY TÍNH VÀ VI XỬ LÝ

ĐỀ TÀI

**PCIE AS INTRA-GPU INTERCONNECT: NGHIÊN CỨU BOTTLENECK
CỦA PCIE TRONG HỆ THỐNG ĐA GPU VÀ MÔ PHỎNG TRÊN VIVADO**

GVHD: PGS.TS Võ Minh Huân

SVTH:

Họ và tên	MSSV
Nguyễn Mai Trung Dũng	24161206
Lê Minh Hải	24161225
Đặng Quang Được	24161217
Lê Quốc Khánh	24161276
Lê Đình Hoàng	24161241
Nguyễn Hữu Đức	24161221

Thành phố Hồ Chí Minh, Tháng 06 năm 2026

DANH SÁCH NHÓM THAM GIA VIẾT BÁO CÁO ĐỀ TÀI

STT	Họ và tên	MSSV	Nội dung thực hiện	Tỷ lệ hoàn thành	Ký tên
1	Nguyễn Mai Trung Dũng	24161206	Tổng hợp tài liệu và viết: - Chương 1: Giới thiệu Viết - Chương 2: Cơ sở lý thuyết (PCIe, Bus, Bottleneck) Chuẩn hóa format báo cáo	100%	
2	Lê Minh Hải	24161225	Thiết kế kiến trúc hệ thống Vẽ sơ đồ: - Sơ đồ bus vs PCIe - Sơ đồ PCIe Switch Viết Chương 3.1: Kiến trúc tổng thể Viết Chương 3.2: Mục tiêu mô phỏng Viết Chương 3.3: Kiến trúc tổng thể của hệ thống	100%	
3	Đặng Quang Được	24161217	Code module: - gpu_sender - pcie_switch Giải thích thuật toán Round-Robin Arbitration Viết Chương 3.4: Thiết kế GPU sender Viết Chương 3.5: Thiết kế PCIe Switch	100%	
4	Lê Quốc Khánh	24161276	Code: - pcie_fifo - gpu_memory	100%	

			Phân tích: - FIFO - Backpressure Viết Chương 3.6: FIFO & Memory Viết Chương 3.7: Thiết kế GPU Receiver		
5	Nguyễn Hữu Đức	24161221	Viết Testbench Chạy mô phỏng Vivado Thu thập: - waveform - số liệu throughput, stall, fifo_count Viết Chương 4: Kết quả mô phỏng	100%	
6	Lê Đình Hoàng	24161241	Phân tích kết quả: - So sánh 1 GPU / 2 GPU / 4 GPU Vẽ biểu đồ: - Throughput - Stall Viết: - Chương 4 (phân tích) - Chương 5: Kết luận & hướng phát triển	100%	

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm:

KÝ TÊN CỦA GIẢNG VIÊN

LỜI CẢM ƠN

Em xin chân thành cảm ơn Thầy/Cô đã tận tình giảng dạy, hướng dẫn và truyền đạt những kiến thức quý báu trong suốt quá trình học tập môn học này. Những kiến thức đó không chỉ giúp em hoàn thành tốt bài báo cáo mà còn là nền tảng quan trọng cho việc học tập và nghiên cứu sau này. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến Thầy Võ Minh Huân đã luôn hỗ trợ, định hướng và góp ý để em có thể hoàn thiện đề tài một cách tốt nhất. Những ý kiến đóng góp của Thầy đã giúp em hiểu rõ hơn về vấn đề nghiên cứu cũng như cải thiện kỹ năng thực hiện đề tài. Em cũng xin cảm ơn bạn bè đã cùng trao đổi, thảo luận và hỗ trợ trong quá trình thực hiện báo cáo. Do kiến thức và thời gian thực hiện còn hạn chế, bài báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của Thầy để bài báo cáo được hoàn thiện hơn. Em xin chân thành cảm ơn!

CHƯƠNG 1: GIỚI THIỆU

1.1. Đặt vấn đề

Trong những năm gần đây, sự phát triển mạnh mẽ của trí tuệ nhân tạo, học sâu và tính toán hiệu năng cao đã làm gia tăng nhu cầu sử dụng các hệ thống đa GPU. Các GPU hiện đại không chỉ cần khả năng tính toán mạnh mẽ mà còn phải trao đổi một lượng lớn dữ liệu với nhau trong quá trình xử lý.

Trong các hệ thống đa GPU, PCI Express (PCIe) là chuẩn kết nối phổ biến nhất hiện nay nhờ khả năng tương thích cao, chi phí thấp và hệ sinh thái rộng lớn. Tuy nhiên, khi số lượng GPU tăng lên, lưu lượng dữ liệu trao đổi giữa các GPU cũng tăng đáng kể, khiến PCIe có thể trở thành nút thắt cổ chai (bottleneck) của toàn hệ thống.

Do đó, việc nghiên cứu đặc điểm hoạt động của PCIe, phân tích các nguyên nhân gây bottleneck và đánh giá ảnh hưởng của bottleneck trong môi trường đa GPU là cần thiết.

1.2. Mục tiêu đề tài

Đề tài được thực hiện với các mục tiêu sau:

- Tìm hiểu kiến trúc và nguyên lý hoạt động của PCI Express.
- Nghiên cứu các thế hệ PCIe hiện nay.
- Phân tích bottleneck của PCIe trong hệ thống đa GPU.
- Xây dựng mô hình mô phỏng giao tiếp giữa nhiều GPU thông qua PCIe.
- Đánh giá hiện tượng nghẽn thông qua các chỉ số như throughput, FIFO occupancy và stall.

1.3. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu PCIe trong vai trò kết nối giữa nhiều GPU trong cùng một hệ thống (Intra-Node GPU Interconnect). Mô hình được xây dựng bằng Verilog HDL và mô phỏng trên Vivado. Đề tài không triển khai PCIe IP Core thực tế mà sử dụng mô hình trừu tượng gồm PCIe Switch, FIFO và GPU Receiver nhằm nghiên cứu hành vi nghẽn của hệ thống.

1.4. Bố cục

Chương 1: Giới thiệu.

Chương 2: Cơ sở lý thuyết.

Chương 3: Thiết kế mô hình mô phỏng.

Chương 4: Kết quả mô phỏng và đánh giá.

Chương 5: Kết luận và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về GPU và hệ thống đa GPU

2.1.1. Tổng quan về GPU

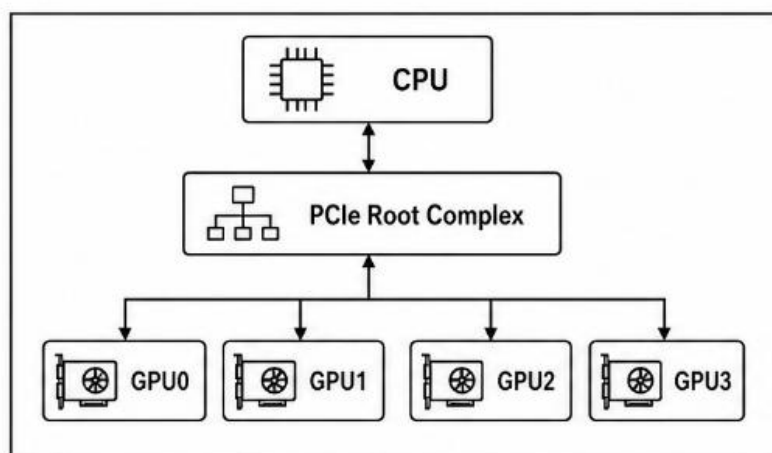
GPU (Graphics Processing Unit) là bộ xử lý chuyên dụng được thiết kế nhằm thực hiện các phép tính song song với số lượng lớn. Ban đầu GPU được sử dụng chủ yếu trong xử lý đồ họa, tuy nhiên với sự phát triển của trí tuệ nhân tạo (Artificial Intelligence - AI), học sâu (Deep Learning) và tính toán hiệu năng cao (High Performance Computing - HPC), GPU ngày càng đóng vai trò quan trọng trong nhiều lĩnh vực nghiên cứu và công nghiệp.

Khác với CPU tập trung vào việc xử lý tuần tự với số lượng lõi ít nhưng hiệu năng trên mỗi lõi cao, GPU được thiết kế với hàng nghìn lõi xử lý nhỏ có khả năng thực hiện đồng thời nhiều phép toán giống nhau trên khối lượng dữ liệu lớn. Điều này giúp GPU đạt hiệu suất vượt trội trong các bài toán tính toán song song.

Hiện nay các GPU hiện đại như NVIDIA H100, NVIDIA A100 hay AMD MI300X được sử dụng rộng rãi trong các trung tâm dữ liệu AI, hệ thống điện toán đám mây và siêu máy tính.

2.1.2 Hệ thống đa GPU

Trong nhiều ứng dụng AI và HPC, một GPU đơn lẻ không còn đáp ứng đủ yêu cầu về năng lực tính toán hoặc dung lượng bộ nhớ. Do đó, nhiều GPU được kết nối trong cùng một hệ thống để cùng thực hiện một tác vụ. Một hệ thống đa GPU điển hình được mô tả như sau:

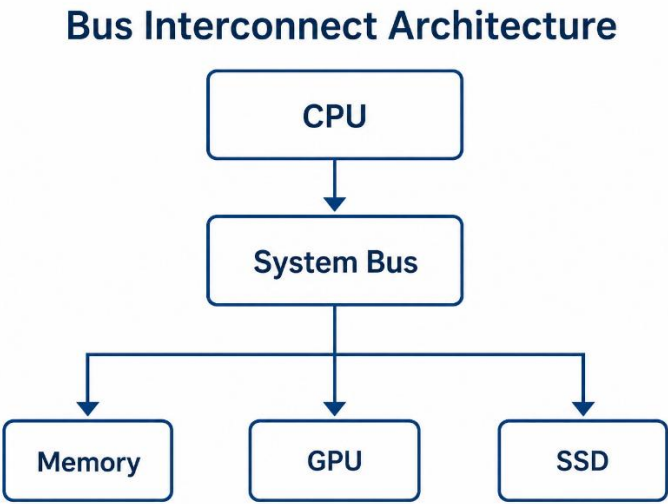


Hình ảnh 2.1: Hệ thống đa GPU điển hình

Trong hệ thống này, các GPU có thể trao đổi dữ liệu với nhau thông qua các công nghệ kết nối như PCI Express (PCIe), NVLink, NVSwitch, CXL và UALink. Trong số đó, PCIe là chuẩn kết nối phổ biến nhất hiện nay nhờ khả năng tương thích rộng rãi và chi phí triển khai thấp. Tuy nhiên, khi số lượng GPU tăng lên, khối lượng dữ liệu trao đổi giữa các GPU cũng tăng nhanh, dẫn đến nguy cơ xuất hiện hiện tượng nghẽn cổ chai trong hệ thống truyền thông.

2.2. Bus Interconnect trong hệ thống máy tính

Bus Interconnect là hệ thống kết nối cho phép CPU, bộ nhớ và các thiết bị ngoại vi trao đổi dữ liệu với nhau. Chức năng chính là truyền dữ liệu, truyền địa chỉ và truyền tín hiệu điều khiển. Mô hình tổng quát:



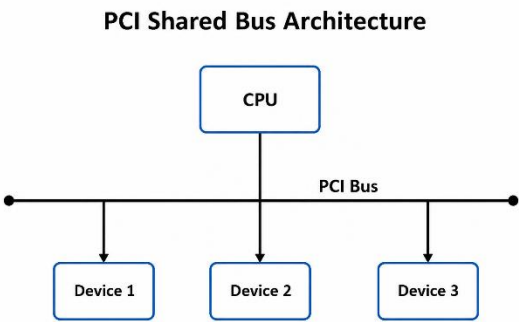
Hình ảnh 2.2: Bus Interconnect

2.2.2. Các thể hệ Bus Interconnect

Chuẩn	Đặc điểm
ISA	Bus đơn đầu
PCI	Bus song song
AGP	Chuyên cho đồ họa
PCI-X	Nâng cấp từ PCI
PCIe	Bus nối tiếp tốc độ cao

2.3. So sánh PCI và PCIe

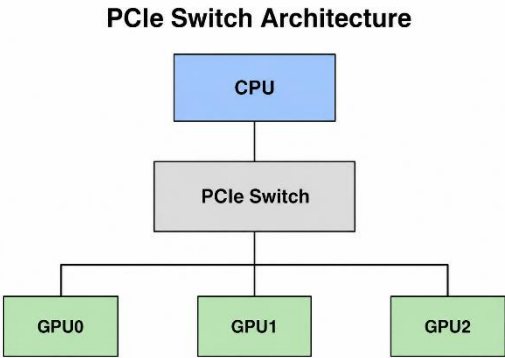
PCI có đặc điểm shared bus và parallel communication. Ví dụ như sau:



Hình ảnh 2.3: Đặc điểm shared bus và parallel communication

→ Nhược điểm: các thiết bị dùng chung bus, dễ xảy ra tranh chấp, khó mở rộng băng thông.

PCIe có đặc điểm: point-to-point và serial communication. Ví dụ như sau:



Hình ảnh 2.4: Các thiết bị dùng chung bus

→ Ưu điểm: băng thông cao hơn, độ trễ thấp hơn, hỗ trợ Full Duplex và mở rộng theo lane.

Bảng so sánh PCI và PCIe

Tiêu chí	PCI	PCIe
Kiểu truyền	Parallel	Serial
Kiến trúc	Shared bus	Point-to-Point
Băng thông	Thấp	Cao
Full Duplex	Không	Có
Khả năng mở rộng	Hạn chế	Tốt
Phù hợp với Multi-GPU	Không có	Có

2.4. Tổng quan về PCI Express

2.4.1. Giới thiệu PCIe

PCI Express (PCIe) là chuẩn giao tiếp nối tiếp tốc độ cao được phát triển nhằm thay thế các chuẩn bus truyền thống như PCI và AGP. PCIe sử dụng kiến trúc kết nối điểm - điểm, cho phép mỗi thiết bị giao tiếp với hệ thống thông qua một liên kết riêng biệt thay vì dùng chung bus như các thế hệ trước. Kiến trúc PCIe mang lại nhiều ưu điểm như:

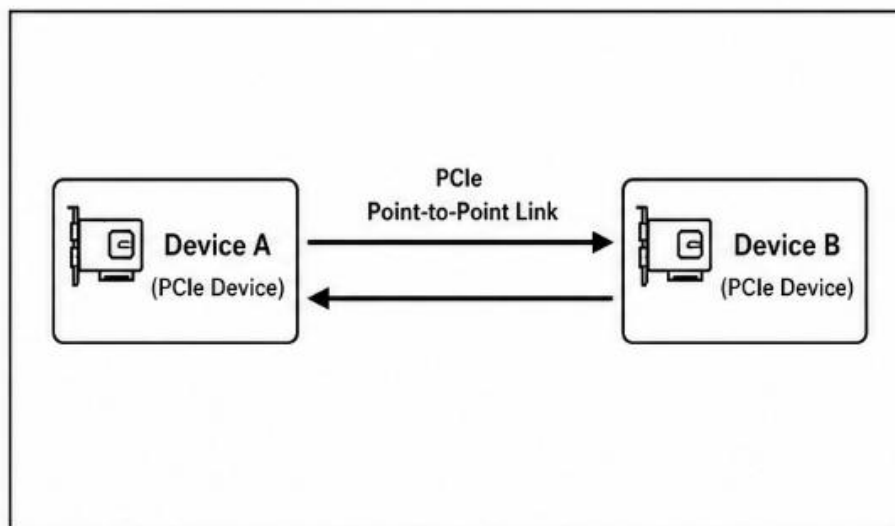
- Tăng băng thông truyền dữ liệu.
- Giảm độ trễ truyền thông.
- Hỗ trợ mở rộng linh hoạt.
- Khả năng tương thích ngược giữa các thế hệ.

PCIe hiện là chuẩn kết nối mặc định cho GPU, FPGA, SSD NVMe, card mạng tốc độ cao và nhiều thiết bị tăng tốc khác.

2.4.2. Đặc điểm của PCIe

Kiến trúc Point-to-Point

PCIe sử dụng mô hình kết nối trực tiếp giữa hai thiết bị.

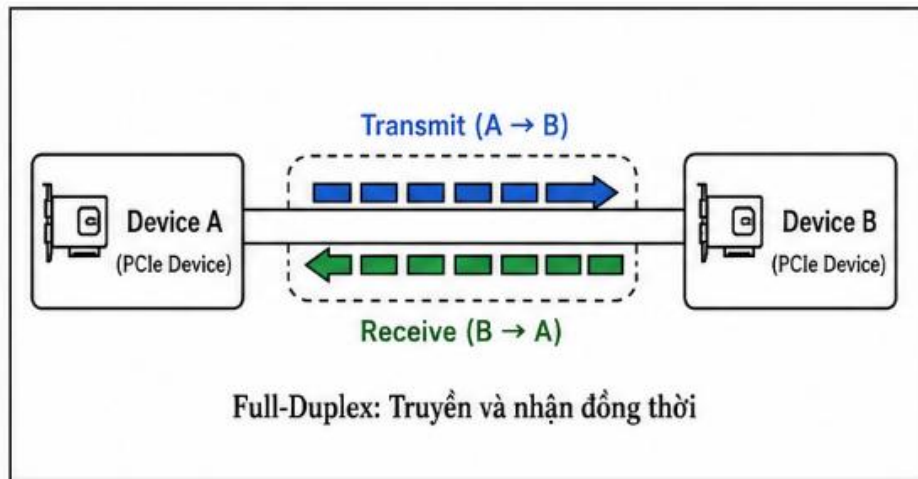


Hình ảnh 2.5. Kiến trúc Point-to-Point

Mỗi liên kết PCIe hoạt động độc lập, giúp giảm tranh chấp tài nguyên và tăng hiệu quả truyền dữ liệu.

Kiến trúc Full-Duplex

PCIe hỗ trợ truyền và nhận dữ liệu đồng thời.



Hình ảnh 2.6: Truyền thông Full-duplex trong PCIe

Điều này giúp tăng hiệu suất sử dụng đường truyền so với các chuẩn Half-Duplex.

Mở rộng theo số Lane

PCIe cho phép ghép nhiều lane để tăng băng thông.

Các cấu hình phổ biến:

Cấu hình	Số lane
PCIe x1	1
PCIe x4	4
PCIe x8	8
PCIe x16	16

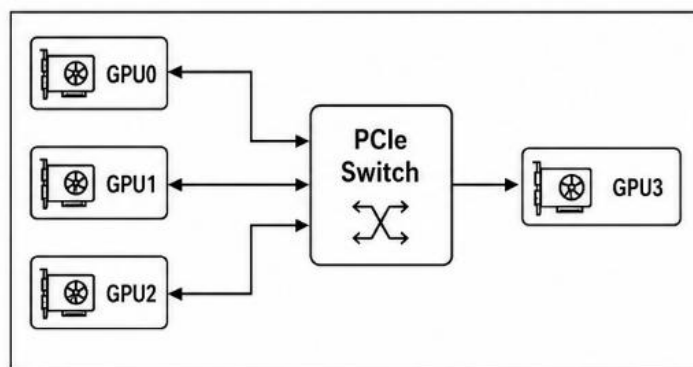
Trong các hệ thống GPU hiện đại, PCIe x16 là cấu hình được sử dụng phổ biến nhất.

2.4.3. Vai trò của PCIe trong hệ thống đa GPU

Trong các hệ thống nhiều GPU, PCIe đóng vai trò là hạ tầng truyền thông giữa:

- CPU và GPU
- GPU và GPU
- GPU và bộ nhớ lưu trữ tốc độ cao
- GPU và các thiết bị tăng tốc khác

Ví dụ:



Hình 2.7: Vai trò của PCIe trong hệ thống đa GPU

Dữ liệu được truyền thông qua PCIe Fabric và PCIe Switch. Khi số lượng GPU tăng lên, PCIe phải xử lý lượng lớn dữ liệu trao đổi đồng thời, dẫn đến nguy cơ bão hòa băng thông và phát sinh hiện tượng nghẽn cổ chai.

2.5. Các thế hệ PCIe

PCIe liên tục được cải tiến nhằm đáp ứng nhu cầu ngày càng cao của các hệ thống tính toán hiện đại.

Thế hệ	Tốc độ truyền (GT/s)
PCIe 1.0	2.5
PCIe 2.0	5
PCIe 3.0	8
PCIe 4.0	16
PCIe 6.0	64

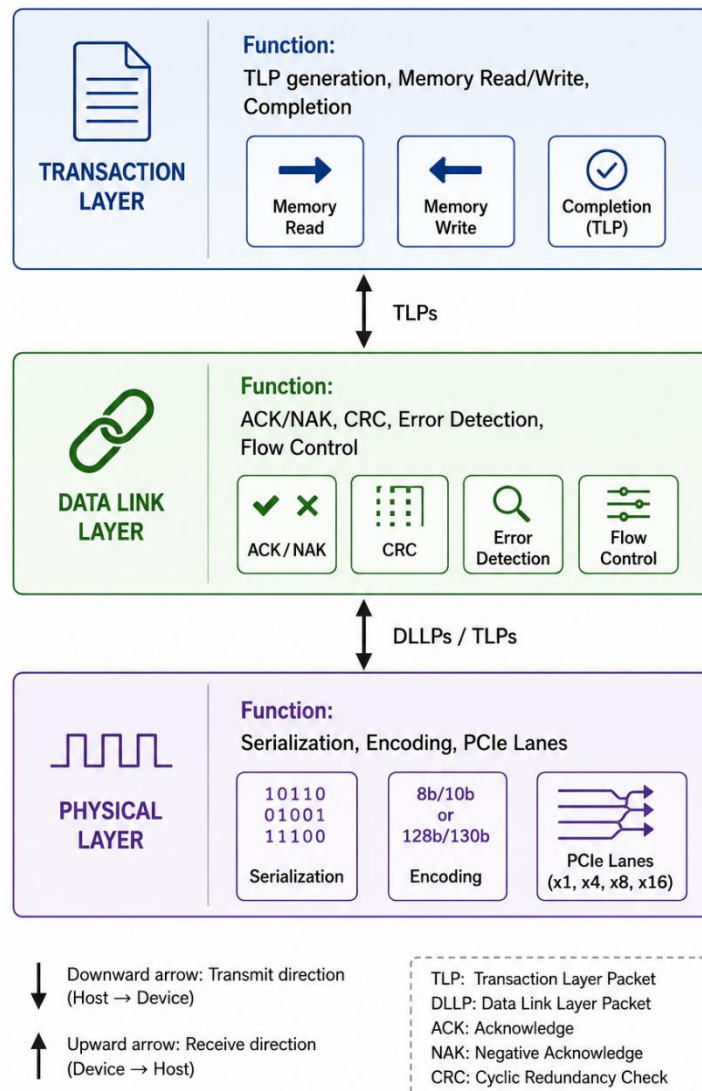
Mỗi thế hệ mới đều tăng gấp đôi băng thông so với thế hệ trước, đồng thời vẫn duy trì khả năng tương thích ngược với các thiết bị PCIe cũ. PCIe 6.0 còn sử dụng kỹ thuật PAM4 và cơ chế FLIT nhằm nâng cao hiệu suất truyền dữ liệu trong các hệ thống AI và HPC thế hệ mới.

2.6. Giao thức PCI Express

PCIe không hoạt động như một bus truyền thống mà sử dụng kiến trúc giao tiếp dạng gói tin (packet-based communication). Mọi dữ liệu trao đổi giữa các thiết bị đều được đóng gói thành các packet trước khi truyền đi. Kiến trúc giao thức PCIe được

chia thành ba lớp chính:

PCI Express (PCIe) Protocol Stack Architecture



Hình 2.8: Kiến trúc giao thức PCIe

2.6.1. Transaction Layer

Transaction Layer là lớp cao nhất trong kiến trúc PCIe. Nhiệm vụ chính của lớp này là:

- Tạo và xử lý các gói tin PCIe
- Quản lý các giao dịch đọc và ghi dữ liệu
- Kiểm soát luồng dữ liệu giữa các thiết bị
- Transaction Layer sử dụng các gói tin gọi là TLP (Transaction Layer Packet)

Một số loại TLP phổ biến:

Loại TLP	Chức năng
Memory Read	Đọc dữ liệu từ bộ nhớ
Memory Write	Ghi dữ liệu vào bộ nhớ
Completion	Phản hồi yêu cầu đọc
Message	Truyền thông tin điều khiển

Ví dụ:

GPU0 muốn gửi dữ liệu cho GPU1

→ Tạo Memory Write TLP

→ Truyền qua PCIe

→ GPU1 nhận và xử lý

2.6.2. Data Link Layer

Data Link Layer nằm giữa Transaction Layer và Physical Layer. Nhiệm vụ chính:

- Kiểm tra lỗi truyền dữ liệu.
- Đảm bảo dữ liệu đến đúng đích.
- Hỗ trợ truyền lại khi xảy ra lỗi.

Data Link Layer sử dụng:

- ACK (Acknowledgement)
- NAK (Negative Acknowledgement)

Ví dụ:

GPU0 gửi packet

GPU1 nhận thành công

→ ACK

GPU1 phát hiện lỗi

→ NAK

GPU0 gửi lại packet

Nhờ cơ chế này, PCIe có độ tin cậy cao trong quá trình truyền dữ liệu.

2.6.3. Physical Layer

Physical Layer là lớp thấp nhất trong giao thức PCIe. Nhiệm vụ:

- Mã hóa dữ liệu.
- Đồng bộ tín hiệu.
- Truyền dữ liệu trên các lane vật lý.

Một lane PCIe gồm: TX+, TX-, RX+, RX-

Mỗi lane hỗ trợ truyền và nhận dữ liệu độc lập

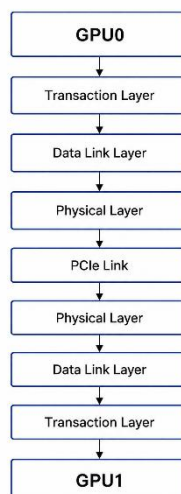
Nhiều lane có thể được ghép lại:

- PCIe x1
- PCIe x4
- PCIe x8
- PCIe x16

để tăng băng thông truyền dữ liệu.

2.6.4. Quá trình truyền dữ liệu trong PCIe

Quá trình truyền dữ liệu giữa hai GPU có thể được mô tả như sau:



Hình ảnh 2.9: Quá trình truyền dữ liệu giữa hai GPU

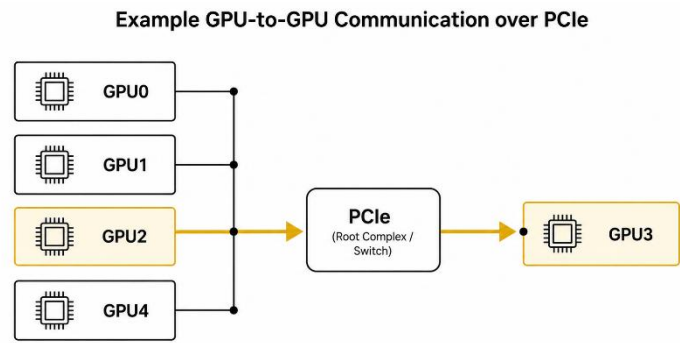
Đầu tiên dữ liệu được đóng gói thành TLP tại Transaction Layer. Sau đó Data Link Layer bổ sung thông tin kiểm tra lỗi và Physical Layer thực hiện truyền dữ liệu trên đường PCIe vật lý.

2.7. Bottleneck của PCIe trong hệ thống đa GPU

2.7.1. Khái niệm Bottleneck

Bottleneck là hiện tượng một thành phần trong hệ thống có tốc độ xử lý thấp hơn các thành phần còn lại, làm giảm hiệu năng tổng thể. Trong hệ thống đa GPU, bottleneck thường xuất hiện khi tốc độ trao đổi dữ liệu không theo kịp tốc độ xử lý của

GPU. Ví dụ như sau:



Hình ảnh 2.10: Ví dụ minh họa về việc trao đổi dữ liệu GPU

Mặc dù các GPU có khả năng tính toán rất cao nhưng dữ liệu vẫn phải truyền qua PCIe. Khi khối lượng dữ liệu quá lớn, PCIe có thể trở thành nút thắt cổ chai của toàn hệ thống.

2.7.2. Giới hạn băng thông

Một trong những nguyên nhân quan trọng nhất gây bottleneck là giới hạn băng thông. Các GPU hiện đại có băng thông bộ nhớ rất lớn. Ví dụ như sau:

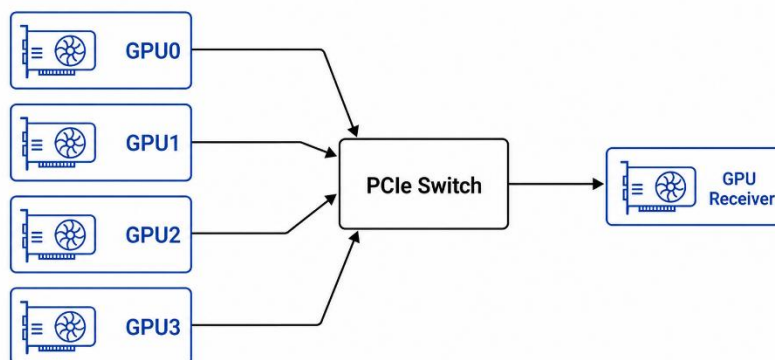
Thành phần	Băng thông
HBM trên GPU hiện đại	Hàng nghìn GB/s
PCIe Gen4 x16	Khoảng 32 GB/s
PCIe Gen5 x16	Khoảng 64 GB/s

Có thể thấy tốc độ xử lý nội bộ của GPU cao hơn rất nhiều so với tốc độ truyền dữ liệu qua PCIe. Điều này khiến GPU phải chờ dữ liệu trong nhiều trường hợp.

2.7.3. Tranh chấp tài nguyên

Khi nhiều GPU cùng sử dụng một liên kết PCIe hoặc một PCIe Switch, hiện tượng tranh chấp tài nguyên sẽ xảy ra. Ví dụ như sau:

\



Hình ảnh 2.11: Hiện tượng tranh chấp tài nguyên trong PCIe

Tại một thời điểm, PCIe Switch chỉ có thể phục vụ một số lượng giao dịch nhất định. Do đó GPU phải chờ lượt truyền làm tăng độ trễ và giảm hiệu suất truyền thông.

2.7.4. Tích tụ dữ liệu trong Buffer

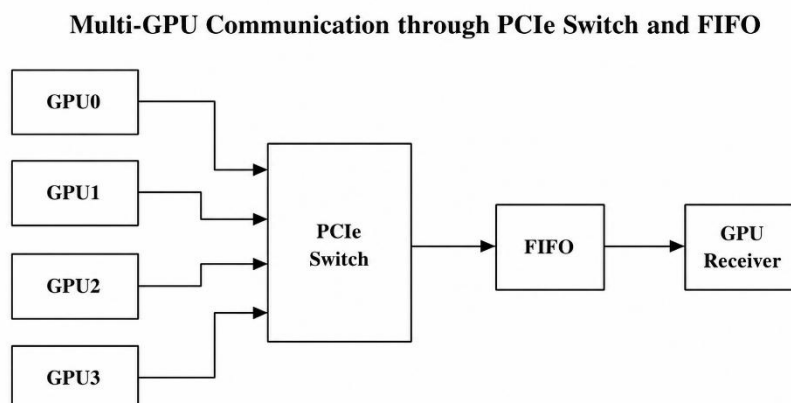
Khi dữ liệu được gửi đến nhanh hơn khả năng xử lý của phía nhận, dữ liệu sẽ bắt đầu tích tụ trong các buffer hoặc FIFO. Hiện tượng này được thể hiện trong mô hình mô phỏng thông qua tín hiệu `fifo_count` tăng. Khi FIFO đầy `fifo_full = 1` hệ thống không thể nhận thêm dữ liệu mới.

2.7.5. Hiện tượng Stall

Stall xảy ra khi GPU muốn gửi dữ liệu nhưng không được cấp quyền truyền. Trong mô hình mô phỏng khi `valid = 1` và `ready = 0`. Khi điều kiện trên xảy ra, GPU phải chờ số chu kỳ chờ được lưu lại trong biến `stall counter` thì giá trị `stall` càng lớn chứng tỏ mức độ nghẽn của hệ thống càng cao.

2.7.6. Liên hệ với mô hình mô phỏng

Trong đề tài này, bottleneck của PCIe được mô phỏng thông qua mô hình:



Hình ảnh 2.12: Bottleneck của PCIe

Hiện tượng nghẽn được đánh giá thông qua:

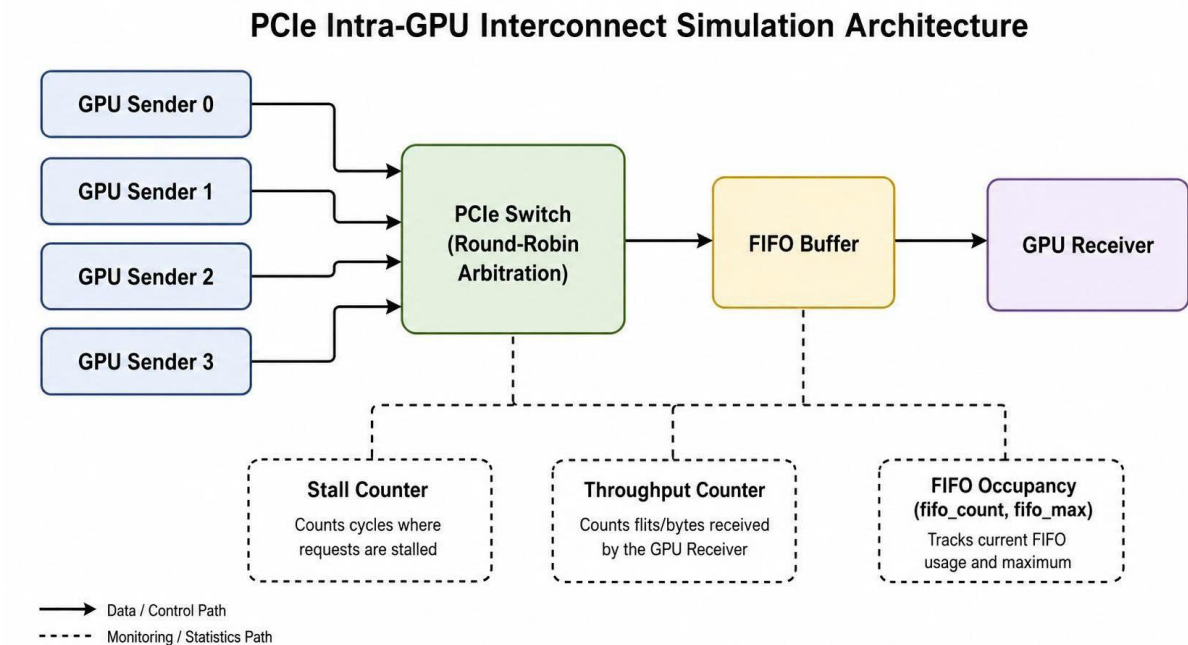
- Stall Counter.
- Throughput Counter.
- FIFO Occupancy (fifo_count).
- FIFO Full (fifo_full).

Khi số lượng GPU tăng từ 1 lên 4, giá trị stall tăng đáng kể và FIFO có xu hướng đầy lên. Điều này cho thấy PCIe Switch và bộ đệm FIFO trở thành điểm nghẽn của hệ thống truyền thông đa GPU.

CHƯƠNG 3: THIẾT KẾ MÔ HÌNH MÔ PHỎNG

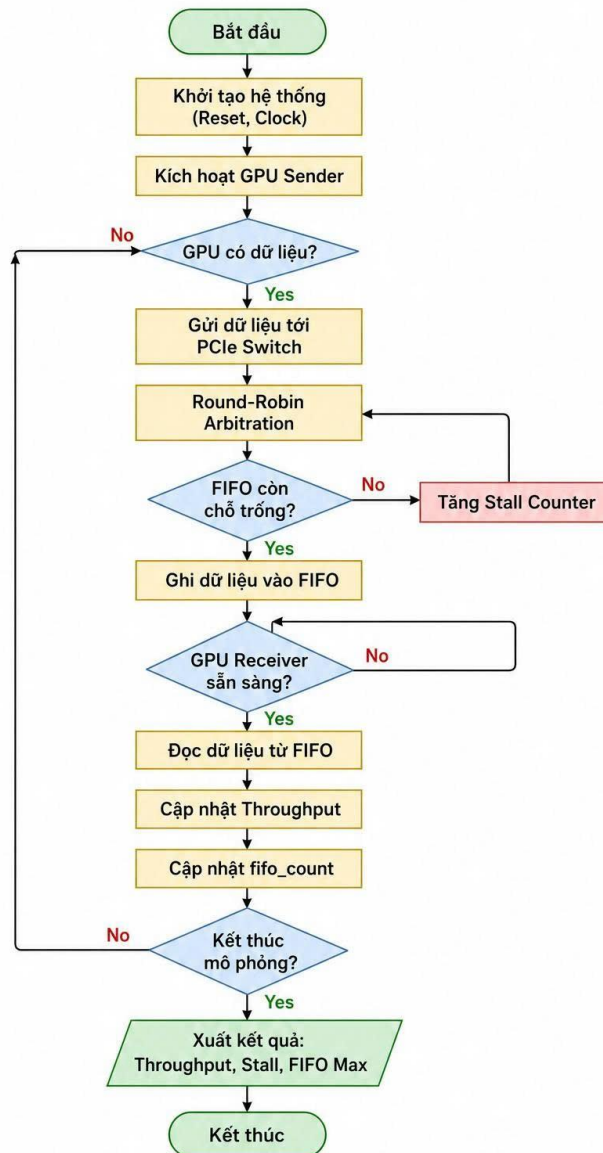
3.1. Kiến trúc tổng thể của hệ thống

Mô hình mô phỏng được xây dựng nhằm nghiên cứu hiện tượng bottleneck của PCIe trong hệ thống đa GPU. Hệ thống gồm các GPU Sender, PCIe Switch, FIFO Buffer và GPU Receiver.

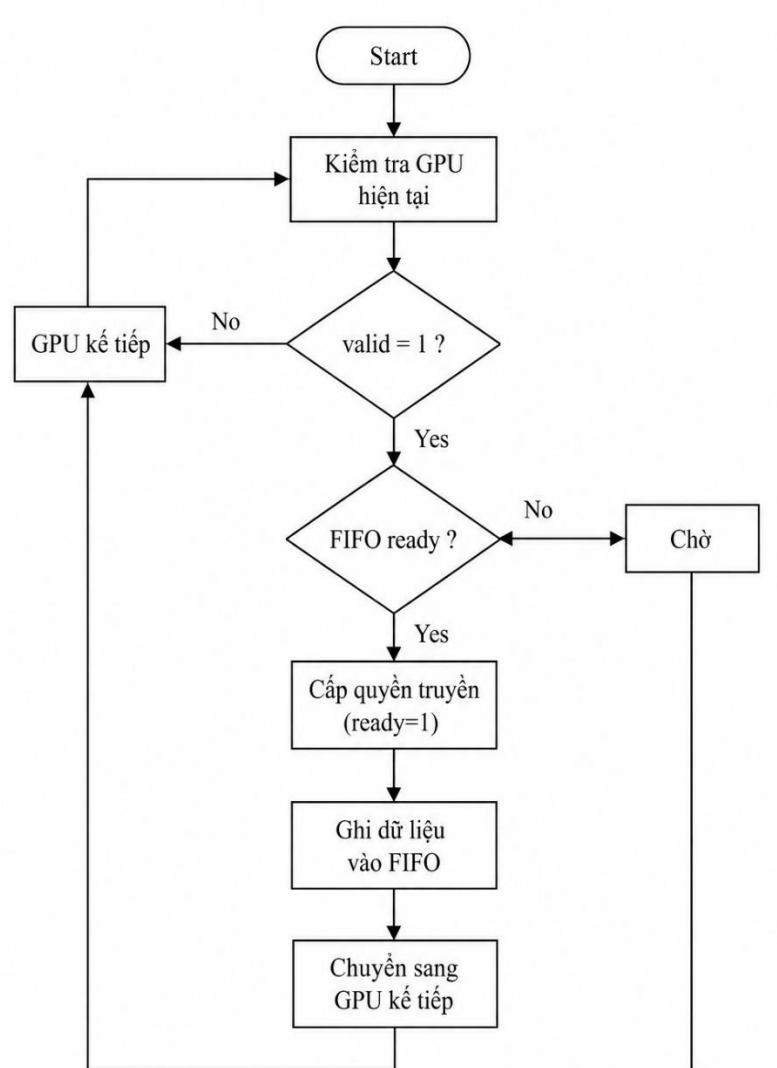


Hình 3.1. Sơ đồ khối tổng thể của mô hình PCIe Intra-GPU Interconnect
Chú thích từng khối:

- GPU Sender: sinh dữ liệu mô phỏng GPU nguồn.
- PCIe Switch: thực hiện Round-Robin Arbitration.
- FIFO: bộ đệm trung gian.
- GPU Receiver: mô phỏng GPU đích nhận dữ liệu.
- Counter: đo throughput, stall và FIFO occupancy.



Hình 3.2. Lưu đồ hoạt động của mô hình PCIe Intra-GPU Interconnect



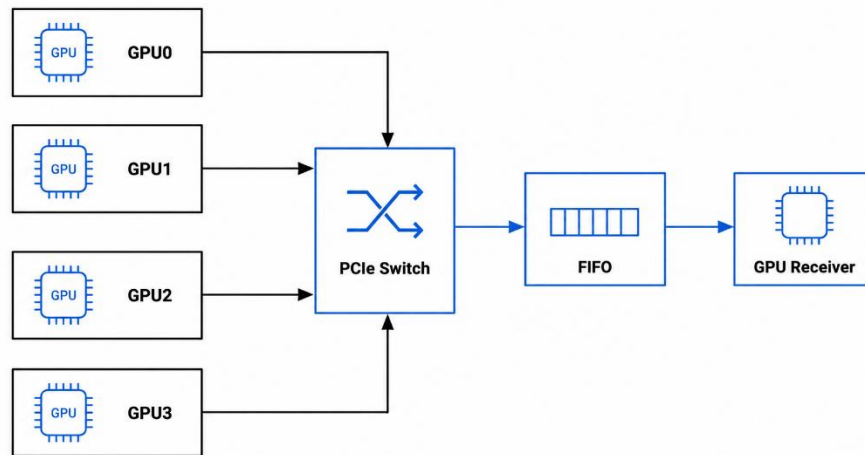
Hình 3.3: Lưu đồ Round-Robin của PCIe Switch

3.2. Mục tiêu mô phỏng

Mục tiêu của mô hình mô phỏng là tái hiện quá trình trao đổi dữ liệu giữa nhiều GPU thông qua kết nối PCIe nhằm nghiên cứu hiện tượng nghẽn cổ chai (bottleneck) khi lưu lượng truyền dữ liệu tăng cao. Thay vì triển khai đầy đủ giao thức PCIe thực tế, đề tài sử dụng một mô hình trừu tượng để tập trung vào các yếu tố gây nghẽn như tranh chấp băng thông, tích tụ dữ liệu trong bộ đệm và hiện tượng chờ truyền dữ liệu. Mô hình được xây dựng bằng Verilog HDL và mô phỏng trên Vivado Simulator.

3.3. Kiến trúc tổng thể của hệ thống

Kiến trúc mô phỏng được xây dựng như Hình 3.2.

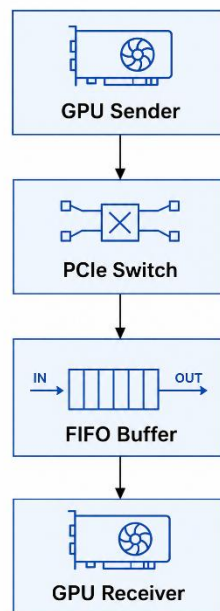


Hình 3.4. Kiến trúc tổng thể của mô hình mô phỏng

Trong mô hình:

- GPU0, GPU1, GPU2 và GPU3 đóng vai trò các nguồn phát dữ liệu.
- PCIe Switch thực hiện phân xử quyền truy cập đường truyền.
- FIFO đóng vai trò bộ đệm dữ liệu trung gian.
- GPU Receiver đóng vai trò thiết bị nhận dữ liệu.

Luồng dữ liệu trong hệ thống được thực hiện theo trình tự:



Hình 3.5. Luồng dữ liệu

3.4. Thiết kế module GPU Sender

3.4.1. Chức năng

GPU Sender được sử dụng để mô phỏng một GPU đang liên tục gửi dữ liệu vào

mạng PCIe. Module này tạo ra các gói dữ liệu tuần tự và chỉ truyền dữ liệu khi được PCIe Switch cấp quyền.

3.4.2. Tín hiệu vào ra

Tín hiệu	Hướng	Chức năng
clk	Input	Clock hệ thống
rst	Input	Reset
enable	Input	Cho phép GPU hoạt động
ready	Input	Tín hiệu cho phép truyền
data	Output	Dữ liệu gửi
valid	Output	Dữ liệu hợp lệ

3.4.3. Nguyên lý hoạt động

Khi:

$$enable = 1$$

$$ready = 1$$

→ GPU sẽ tăng giá trị dữ liệu và gửi tới PCIe Switch.

Ngược lại:

$$enable = 1$$

$$ready = 0$$

→ GPU muốn gửi dữ liệu nhưng chưa được cấp quyền.

Trạng thái này được gọi là:

STALL

và được sử dụng để đánh giá mức độ nghẽn của hệ thống.

3.5. Thiết kế module PCIe Switch

3.5.1 Chức năng

PCIe Switch là thành phần trung tâm của mô hình. Nhiệm vụ chính:

- Nhận dữ liệu từ nhiều GPU.
- Thực hiện phân xử quyền truy cập.

- Chuyển dữ liệu tới FIFO.

3.5.2. Thuật toán phân xử

Trong đề tài sử dụng cơ chế Round-Robin Arbitration. PCIe Switch sẽ cấp quyền truyền theo thứ tự: GPU0 → GPU1 → GPU2 → GPU3 → GPU0 →... Nhờ đó tất cả GPU đều có cơ hội sử dụng đường truyền.

3.5.3. Ý nghĩa trong mô hình

PCIe Switch mô phỏng hiện tượng Bandwidth Sharing tồn tại trong các hệ thống đa GPU thực tế. Khi nhiều GPU cùng gửi dữ liệu chỉ một GPU được phục vụ tại một thời điểm dẫn đến tranh chấp tài nguyên và tăng độ trễ truyền dữ liệu.

3.6. Thiết kế module FIFO

3.6.1. Chức năng

FIFO (First In First Out) được sử dụng như bộ đệm trung gian giữa PCIe Switch và GPU Receiver. FIFO cho phép:

- Lưu trữ tạm thời dữ liệu.
- Giảm mất dữ liệu khi tốc độ gửi lớn hơn tốc độ nhận.
- Quan sát hiện tượng tích tụ dữ liệu trong hệ thống.

3.6.2. Các tín hiệu chính

Tín hiệu	Chức năng
fifo_count	Số phần tử hiện có trong FIFO
fifo_full	FIFO đầy
fifo_empty	FIFO rỗng
fifo_in_ready	FIFO có thể nhận thêm dữ liệu
fifo_out_valid	FIFO có dữ liệu để xuất

3.6.3. Ý nghĩa đối với nghiên cứu bottleneck

Khi tốc độ gửi > tốc độ nhận số lượng phần tử trong FIFO sẽ tăng dần. Điều này được thể hiện thông qua fifo_count tăng khi FIFO đạt dung lượng tối đa fifo_full = 1. PCIe Switch không thể tiếp tục ghi dữ liệu. Đây là biểu hiện rõ ràng của hiện tượng

nghe trong hệ thống truyền thông.

3.7. Thiết kế module GPU Receiver

3.7.1. Chức năng

GPU Receiver mô phỏng GPU đích nhận dữ liệu. Trong mô hình này, GPU Receiver được cấu hình xử lý dữ liệu chậm hơn tốc độ gửi nhằm tạo ra áp lực cho hệ thống truyền thông.

3.7.2. Nguyên lý hoạt động

Receiver chỉ cho phép nhận dữ liệu sau một số chu kỳ clock xác định. Ví dụ: 1 lần nhận / 16 chu kỳ clock. Do đó, $\text{input rate} > \text{output rate}$ khiến dữ liệu tích tụ trong FIFO.

3.7.3. Vai trò trong mô phỏng

Module này được sử dụng để tạo ra hiện tượng backpressure, giống như trong các hệ thống PCIe thực tế khi thiết bị đích xử lý dữ liệu không đủ nhanh.

3.8. Các chỉ số đánh giá hiệu năng

Để đánh giá mức độ nghẽn của hệ thống, đề tài sử dụng ba chỉ số chính.

3.8.1. Stall Counter

Stall Counter ghi nhận số chu kỳ GPU phải chờ trước khi được cấp quyền truyền dữ liệu. Điều kiện tăng bộ đếm:

$$\text{valid} = 1$$

$$\text{ready} = 0$$

Giá trị stall càng lớn chứng tỏ mức độ nghẽn càng cao.

3.8.2. Throughput Counter

Throughput Counter ghi nhận số lượng gói dữ liệu được truyền thành công tới GPU Receiver. Điều kiện ghi nhận

$$\text{fifo_out_valid} \ \&\& \ \text{mem_ready}$$

Thông số này phản ánh hiệu quả truyền dữ liệu của hệ thống.

3.8.3. FIFO Occupancy

FIFO Occupancy được xác định thông qua `fifo_count` và `fifo_max`. Trong đó:

- `fifo_count`: số phần tử hiện tại.
- `fifo_max`: giá trị lớn nhất đạt được trong quá trình mô phỏng.

FIFO Occupancy cho phép đánh giá mức độ tích tụ dữ liệu trong hệ thống.

3.9. Kết luận chương

Trong chương này, đề tài đã trình bày thiết kế của mô hình mô phỏng PCIe Intra-GPU Interconnect gồm bốn GPU Sender, một PCIe Switch, một FIFO và một GPU Receiver. Các bộ đếm Stall, Throughput và FIFO Occupancy được bổ sung nhằm đánh giá hiệu năng truyền dữ liệu và mức độ nghẽn của hệ thống. Mô hình này là cơ sở để thực hiện các thí nghiệm mô phỏng và phân tích bottleneck của PCIe trong Chương 4.

CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ

4.1 Môi trường mô phỏng

Mô hình được xây dựng bằng ngôn ngữ mô tả phần cứng Verilog HDL và được mô phỏng trên phần mềm Vivado Design Suite.

Cấu hình môi trường

Thành phần	Công cụ
Ngôn ngữ mô tả	Verilog HDL
Môi trường phát triển	Vivado
Trình mô phỏng	XSim
Mô hình nghiên cứu	PCIe Intra-GPU Interconnect
Số lượng GPU mô phỏng	4 GPU

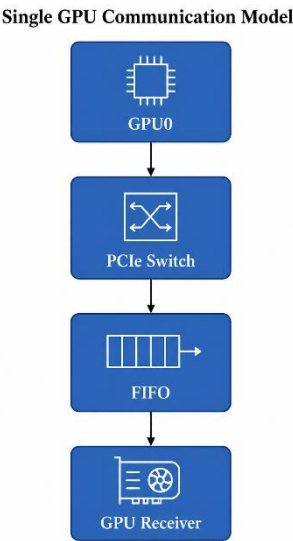
Mục tiêu của mô phỏng là đánh giá ảnh hưởng của số lượng GPU hoạt động đồng thời lên khả năng truyền dữ liệu của PCIe.

4.2. Các testcase thử nghiệm

Để đánh giá mức độ nghẽn của PCIe, ba testcase được thực hiện.

4.2.1. Testcase 1 – một GPU truyền dữ liệu

Trong testcase đầu tiên chỉ có GPU0 hoạt động.



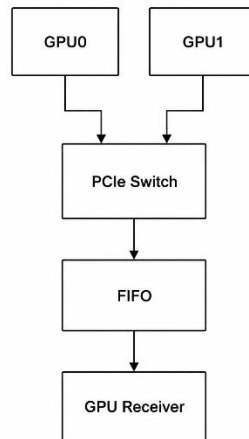
Hình ảnh 4.1: Một GPU truyền dữ liệu

Mục tiêu:

- Thiết lập đường cơ sở (baseline).
- Đo throughput tối đa của hệ thống.
- Quan sát mức sử dụng FIFO khi không có tranh chấp tài nguyên.

4.2.2. Testcase 2 – hai GPU truyền dữ liệu

Trong testcase thứ hai:



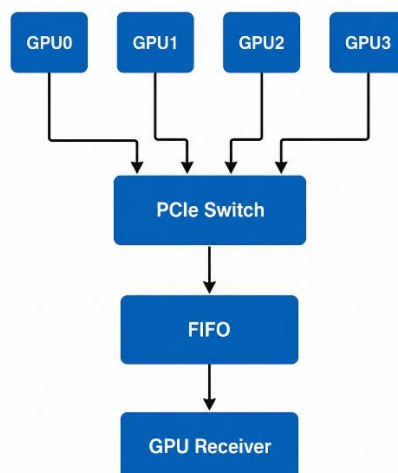
Hình ảnh 4.2: Hai GPU truyền dữ liệu

Mục tiêu:

- Quan sát hiện tượng tranh chấp đường truyền.
- Đánh giá mức tăng của stall counter.
- So sánh với trường hợp một GPU.

4.2.3. Testcase 3 – bốn GPU truyền dữ liệu

Trong testcase cuối cùng:



Hình ảnh 4.3: Bốn GPU truyền dữ liệu

Mục tiêu:

- Tạo áp lực tối đa lên PCIe Switch.
- Quan sát sự tích tụ dữ liệu trong FIFO.
- Đánh giá bottleneck của hệ thống.

4.3. Kết quả mô phỏng Waveform

4.3.1 Quan sát tín hiệu truyền dữ liệu

Các tín hiệu quan trọng trong mô phỏng bao gồm:

- Biểu diễn yêu cầu gửi dữ liệu từ các GPU: v0, v1, v2, v3
- Biểu diễn quyền truyền dữ liệu được PCIe Switch cấp phát: r0, r1, r2, r3

Quan sát waveform cho thấy:

- Các tín hiệu valid luôn ở mức cao.
- Các tín hiệu ready xuất hiện luân phiên.

Điều này chứng minh PCIe Switch đang thực hiện cơ chế phân xử Round-Robin.

4.3.2. Quan sát FIFO

Các tín hiệu FIFO được theo dõi gồm:

fifo_count
fifo_full
fifo_empty
fifo_in_ready
fifo_out_valid

Kết quả cho thấy:

- Khi số GPU tăng, *fifo_count* tăng nhanh hơn.
- FIFO chứa nhiều dữ liệu hơn theo thời gian.
- Một số thời điểm FIFO tiến gần trạng thái đầy.

Điều này chứng minh tốc độ dữ liệu đi vào lớn hơn tốc độ xử lý của GPU

Receiver.

4.3.3. Quan sát Stall Counter

Stall Counter được xây dựng để ghi nhận số chu kỳ mà GPU phải chờ trước khi được truyền dữ liệu.

Điều kiện stall:

valid = 1

$$ready = 0$$

Khi đó:

$$stall = stall + 1$$

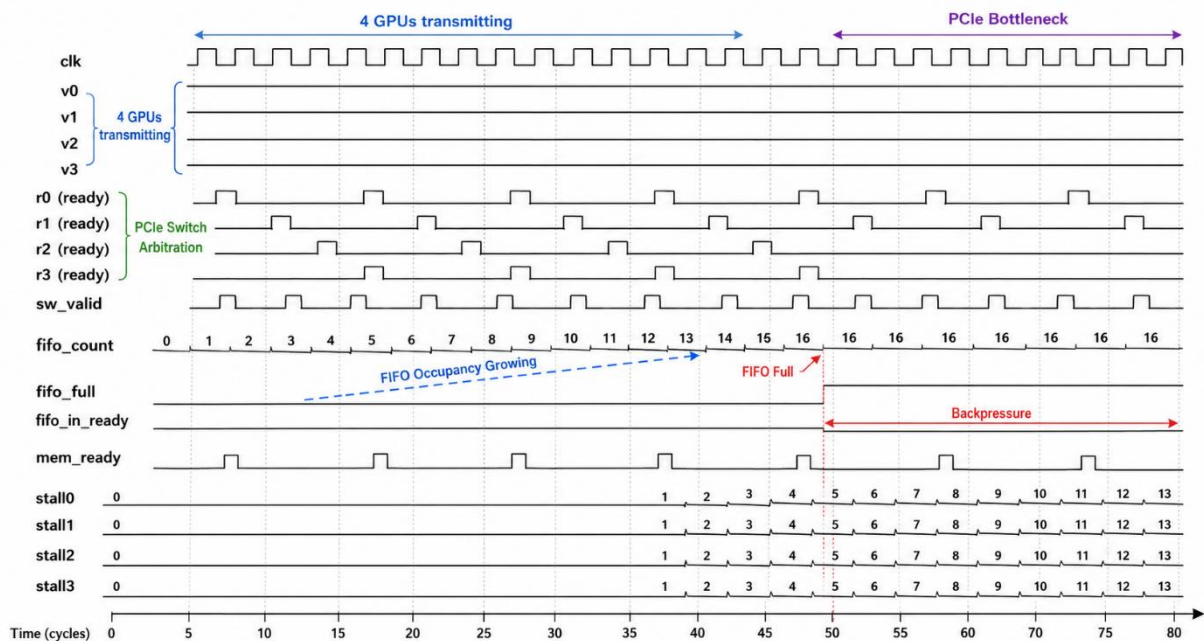
Quan sát waveform cho thấy:

- Stall tăng chậm ở Case 1.
- Stall tăng nhanh hơn ở Case 2.
- Stall tăng mạnh ở Case 3.

Điều này phản ánh mức độ tranh chấp tài nguyên ngày càng lớn khi số lượng GPU tăng lên.

4.4. Phân tích Timing Diagram

Hình Timing Diagram mô tả quá trình dữ liệu được truyền từ nhiều GPU tới GPU Receiver thông qua PCIe Switch và FIFO.



Hình 4.4. Timing Diagram

Giai đoạn 1: GPU gửi dữ liệu. Các tín hiệu $v0 = v1 = v2 = v3 = 1$ cho thấy tất cả GPU đều liên tục phát dữ liệu.

Giai đoạn 2: PCIe Switch phân xử. PCIe Switch cấp quyền truyền lần lượt GPU0 → GPU1 → GPU2 → GPU3 thông qua các tín hiệu r0, r1, r2, r3 tại một thời điểm chỉ một GPU được phép truyền.

Giai đoạn 3: Dữ liệu tích tụ trong FIFO

Do GPU Receiver xử lý chậm hơn tốc độ gửi dữ liệu input Rate > output Rate nên fifo_count tăng dần. Đây là dấu hiệu đầu tiên của hiện tượng nghẽn.

Giai đoạn 4: Xuất hiện Bottleneck. Khi FIFO gần đầy fifo_count → fifo_max PCIe Switch bắt đầu bị hạn chế khả năng truyền dữ liệu.

Kết quả: stall tăng và throughput trên mỗi GPU giảm

4.5. Kết quả định lượng

Sau khi mô phỏng hoàn tất, các giá trị Throughput, FIFO Max và Stall được ghi nhận.

Chỉ số	Case 1 (1 GPU)	Case 2 (2 GPU)	Case 3 (4 GPU)
Throughput	3 packets	6 packets	9 packets
FIFO Max	9	16	16
Stall GPU0	37	82	130
Stall GPU1	0	44	92
Stall GPU2	0	0	47
Stall GPU3	0	0	47

4.6. Đánh giá Bottleneck của PCIe

Từ các kết quả mô phỏng có thể rút ra một số nhận xét quan trọng:

Trường hợp 1 GPU

- Hầu như không có tranh chấp tài nguyên.
- FIFO chỉ chứa một lượng nhỏ dữ liệu.
- Stall ở mức thấp.

→ Hệ thống hoạt động ổn định.

Trường hợp 2 GPU

- Bắt đầu xuất hiện cạnh tranh đường truyền.
- Stall tăng so với Case 1.
- FIFO Occupancy tăng.

→ PCIe Switch bắt đầu trở thành tài nguyên dùng chung.

Trường hợp 4 GPU

- Tất cả GPU cùng tranh chấp quyền truyền dữ liệu.
 - FIFO tích tụ dữ liệu liên tục.
 - Stall tăng mạnh.
 - Throughput trung bình trên mỗi GPU giảm.
- Đây là trường hợp thể hiện rõ nhất hiện tượng bottleneck.

4.7. Kết luận chương

Chương này đã trình bày kết quả mô phỏng hệ thống PCIe Intra-GPU Interconnect bằng Verilog HDL trên Vivado. Kết quả cho thấy khi số lượng GPU tăng lên, hiện tượng tranh chấp tài nguyên tại PCIe Switch xuất hiện ngày càng rõ rệt. Dữ liệu bị tích tụ trong FIFO, dẫn đến sự gia tăng của Stall Counter và giảm hiệu quả truyền dữ liệu. Các kết quả thu được chứng minh rằng PCIe có thể trở thành nút thắt cổ chai trong các hệ thống đa GPU khi khối lượng trao đổi dữ liệu vượt quá khả năng xử lý của đường truyền và bộ đệm trung gian. Đây cũng là lý do các công nghệ kết nối chuyên dụng như NVLink, NVSwitch và UALink được phát triển để thay thế hoặc hỗ trợ PCIe trong các hệ thống AI và HPC hiện đại.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong đề tài này, nhóm đã tiến hành nghiên cứu chuẩn giao tiếp PCI Express (PCIe) trong vai trò kết nối giữa các GPU trong hệ thống đa GPU (Multi-GPU System). Bên cạnh việc tìm hiểu kiến trúc, nguyên lý hoạt động và các thể hệ PCIe, đề tài còn tập trung phân tích các nguyên nhân dẫn đến hiện tượng nghẽn cổ chai (bottleneck) trong quá trình trao đổi dữ liệu giữa các GPU.

Để đánh giá bottleneck của PCIe, nhóm đã xây dựng một mô hình mô phỏng bằng Verilog HDL trên môi trường Vivado. Mô hình gồm bốn GPU Sender, một PCIe Switch, một FIFO đóng vai trò bộ đệm trung gian và một GPU Receiver. Hệ thống được thiết kế nhằm mô phỏng tình huống nhiều GPU cùng gửi dữ liệu thông qua một tài nguyên truyền thông dùng chung.

Thông qua các kịch bản mô phỏng với số lượng GPU hoạt động khác nhau, nhóm đã quan sát và phân tích các chỉ số như:

- Throughput.
- Stall Counter.
- FIFO Occupancy.
- FIFO Full Status.

Kết quả mô phỏng cho thấy khi số lượng GPU tăng lên, lượng dữ liệu truyền vào hệ thống tăng nhanh hơn khả năng xử lý của thiết bị nhận. Điều này làm dữ liệu tích tụ trong FIFO, thể hiện qua sự gia tăng của tín hiệu `fifo_count`. Đồng thời số chu kỳ chờ truyền dữ liệu (stall) cũng tăng lên đáng kể do các GPU phải cạnh tranh quyền truy cập PCIe Switch.

Kết quả thu được chứng minh rằng PCIe có thể trở thành nút thắt cổ chai trong các hệ thống đa GPU khi lưu lượng trao đổi dữ liệu vượt quá khả năng xử lý của liên kết truyền thông và bộ đệm trung gian.

Như vậy, các mục tiêu đề ra ban đầu của đề tài đã được hoàn thành, bao gồm:

- Nghiên cứu kiến trúc PCIe.
- Tìm hiểu các thể hệ PCIe.
- Phân tích giao thức hoạt động của PCIe.
- Xác định các nguyên nhân gây bottleneck trong hệ thống đa GPU.

- Xây dựng mô hình mô phỏng PCIe Intra-GPU Interconnect trên Vivado.
- Đánh giá mức độ nghẽn thông qua các chỉ số định lượng.

5.2. Hạn chế của đề tài

Mặc dù đã mô phỏng được hiện tượng bottleneck của PCIe, đề tài vẫn còn một số hạn chế nhất định.

Thứ nhất, mô hình được xây dựng ở mức hành vi (behavioral level) và chưa triển khai đầy đủ giao thức PCIe thực tế.

Thứ hai, mô hình chưa mô phỏng các thành phần quan trọng của PCIe như

- Transaction Layer Packet (TLP).
- Data Link Layer.
- ACK/NAK Mechanism.
- Credit-Based Flow Control.
- DMA Engine.

Thứ ba, số lượng GPU và lưu lượng truyền dữ liệu trong mô hình còn đơn giản so với các hệ thống AI hoặc HPC hiện đại.

Do đó, kết quả mô phỏng chủ yếu mang tính minh họa và nghiên cứu nguyên lý hoạt động của bottleneck trong PCIe.

5.3. Hướng phát triển

Trong tương lai, đề tài có thể được mở rộng theo các hướng sau:

5.3.1. Mô phỏng giao thức PCIe chi tiết hơn

Xây dựng mô hình mô phỏng gần với PCIe thực tế bằng cách bổ sung:

- Transaction Layer Packet (TLP).
- Data Link Layer.
- ACK/NAK.
- Credit-Based Flow Control.

Điều này giúp mô hình phản ánh chính xác hơn hành vi của PCIe trong thực tế.

5.3.2. Mở rộng quy mô hệ thống đa GPU

Tăng số lượng GPU trong mô hình từ 4 GPU → 8 GPU → 16 GPU để đánh giá mức độ ảnh hưởng của bottleneck khi quy mô hệ thống tăng lên.

5.3.3. So sánh các thế hệ PCIe

Thực hiện mô phỏng và đánh giá PCIe Gen3, PCIe Gen4, PCIe Gen5 và PCIe

Gen6. Thông qua các chỉ số Throughput, Latency, Stall và FIFO Occupancy.

Qua đó xác định hiệu quả cải thiện của từng thế hệ PCIe.

5.3.4. So sánh PCIe với các công nghệ kết nối mới

Thực hiện nghiên cứu và mô phỏng so sánh giữa: PCIe, NVLink, NVSwitch, CXL và UALink. Mục tiêu là đánh giá khả năng giảm bottleneck trong các hệ thống AI và HPC thế hệ mới.

5.3.5 Triển khai trên FPGA

Một hướng phát triển quan trọng là triển khai mô hình trên FPGA sử dụng Xilinx Vivado, PCIe IP Core và AXI Interconnect. Điều này cho phép đánh giá hiệu năng của hệ thống trong môi trường phần cứng thực tế thay vì chỉ mô phỏng bằng phần mềm.

5.4 Tổng kết

Đề tài đã nghiên cứu thành công bottleneck của PCIe trong hệ thống đa GPU thông qua mô hình mô phỏng được xây dựng trên Vivado. Kết quả cho thấy khi nhiều GPU cùng chia sẻ tài nguyên truyền thông, hiện tượng tranh chấp băng thông, tích tụ dữ liệu trong FIFO và gia tăng Stall Counter xuất hiện rõ rệt. Những kết quả này góp phần làm rõ các hạn chế của PCIe trong các hệ thống tính toán hiệu năng cao hiện nay và là cơ sở cho các nghiên cứu sâu hơn về các công nghệ liên kết GPU thế hệ mới

